
TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

Programación 2

TRABAJO PRÁCTICO INTEGRADOR

Food Store

Sistema de Gestión

de Pedidos de Comida (Consola)



JUNIO 2026

Victoria Lugo

Néstor Gómez

Laura Domínguez

Índice

1. Carátula	1
2. Índice	2
3. Marco Teórico	3
3.1. Java	3
3.2. Programación Orientada a Objetos (POO)	3
3.3. Colecciones en Java	3
3.4. Manejo de Excepciones	4
3.5. JDBC (alcance del proyecto)	4
4. Decisiones Técnicas y Arquitectura	5
4.1. Estructura de Paquetes	5
4.2. Diagrama de Clases (UML)	6
4.3. Decisiones de Diseño	7
4.4. Capturas del Sistema Funcionando	8
5. Dificultades y Soluciones	12
6. Bibliografía / Webgrafía	13
7. Anexos	14
7.1. Link al Repositorio GitHub	14
7.2. Link al Video Demostrativo	14

Marco Teórico:

3.1. Java

Java es un lenguaje de programación de propósito general, orientado a objetos y multiplataforma. Para este proyecto se utilizó Java 21, aprovechando sus características modernas como la sintaxis mejorada y las APIs de colecciones.

El sistema se ejecuta en consola, interactuando con el usuario mediante un menú que permite realizar operaciones CRUD sobre las entidades del dominio.

3.2. Programación Orientada a Objetos (POO)

POO es un paradigma de programación que organiza el código en torno a "objetos" que contienen datos (atributos) y comportamientos (métodos).

En este proyecto se aplicaron los siguientes puntos:

Encapsulamiento: Los atributos de las clases son privados y se accede a ellos mediante getters y setters.

Herencia: Todas las entidades (Categoria, Producto, Usuario, Pedido, DetallePedido) heredan de una clase abstracta Base que contiene los atributos comunes: id, eliminado y createdAt.

Polimorfismo: Se implementó la interfaz Calculable, que define el método calcularTotal(). La clase Pedido implementa esta interfaz, permitiendo que se trate de manera genérica.

Abstracción: La clase Base es abstracta, por lo que no puede instanciarse directamente.

3.3. Colecciones en Java

Las colecciones son estructuras de datos dinámicas que permiten almacenar grupos de objetos. En este proyecto se utilizaron:

ArrayList<T>: Para almacenar listas de entidades (categorías, productos, usuarios, pedidos, detalles).

HashMap<Long, T>: Opcional, para búsqueda rápida por ID.

El almacenamiento es en memoria, lo que significa que los datos se pierden al finalizar la ejecución del programa.

3.4. Manejo de Excepciones

Se crearon excepciones personalizadas para manejar errores específicos del dominio:

`EntidadNoEncontradaException`: Cuando se busca un ID que no existe o está eliminado.

`StockInvalidoException`: Cuando se intenta crear un producto con stock negativo.

`PrecioInvalidoException`: Cuando se intenta crear un producto con precio negativo.

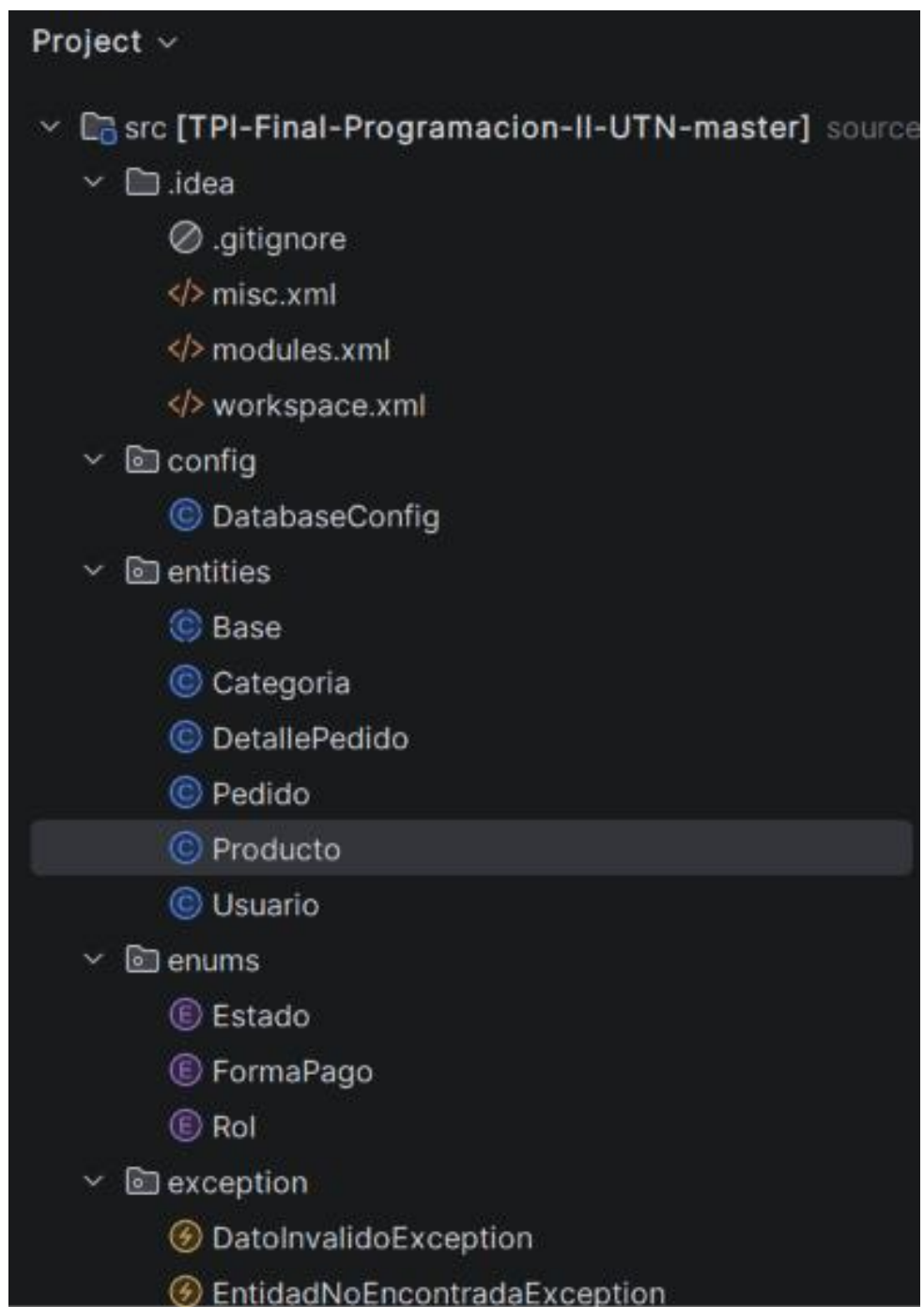
`MailDuplicadoException`: Cuando se intenta registrar un usuario con un mail ya existente.

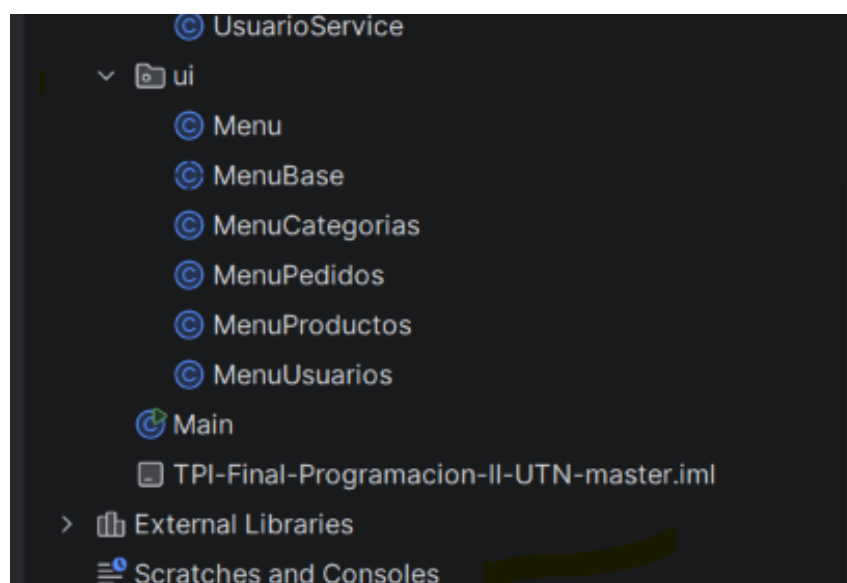
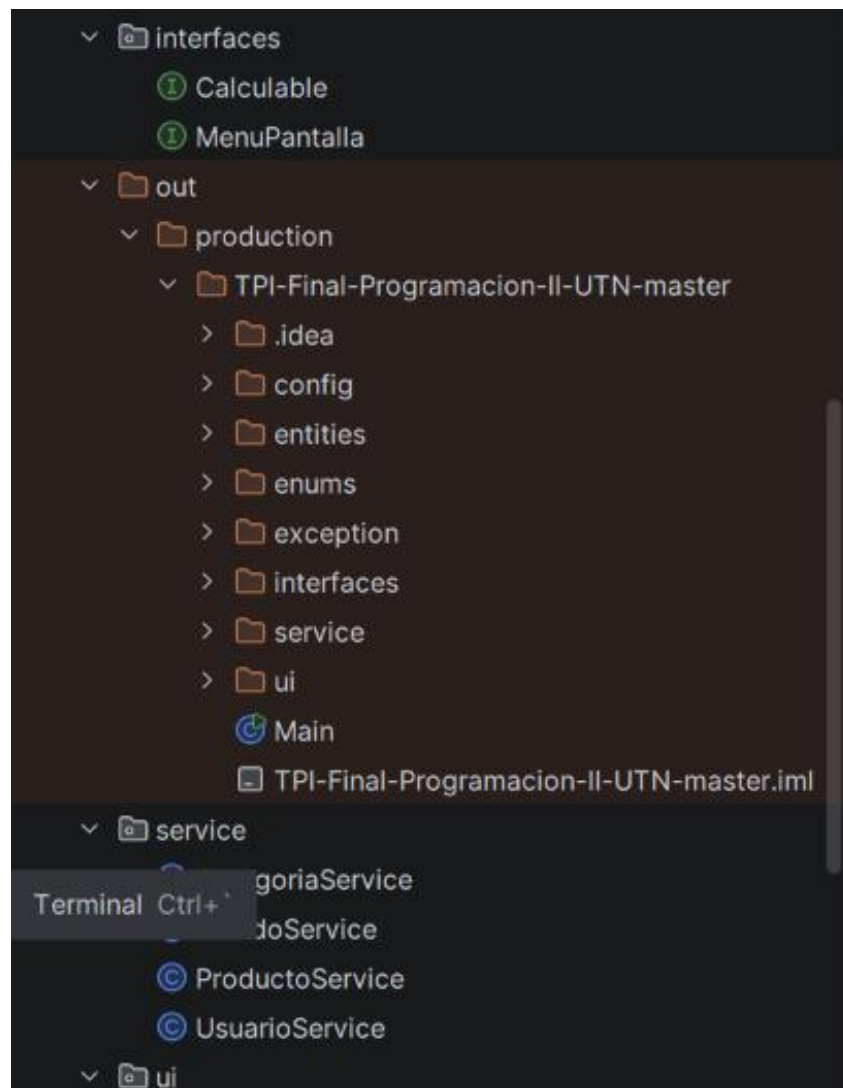
Todas las operaciones están envueltas en bloques try-catch para evitar cierres inesperados del programa.

3.5. JDBC (alcance del proyecto)

Según la consigna, este proyecto NO utiliza base de datos; el almacenamiento es en memoria mediante colecciones. Sin embargo, el nombre "Food Store" y la mención de JDBC en el enunciado indican que podría extenderse en el futuro para persistir los datos en una base de datos relacional.

4.1. Estructura de Paquetes





Justificación de la arquitectura:

Entities: Contienen el modelo de dominio y la lógica de negocio básica.

Services: Coordinan los casos de uso, aplican validaciones y reglas de negocio.

Menu: Solo se encargan de la interacción con el usuario (entrada/salida por consola).

Exceptions: Centralizan el manejo de errores personalizados.

4.2. Diagrama de Clases (UML)

El siguiente diagrama de clases UML fue elaborado como guía para el desarrollo del sistema Food Store. En él se modelan todas las entidades necesarias para la gestión de pedidos, junto con sus atributos, métodos y relaciones.

Para mantener una estructura ordenada y evitar redundancias, creamos una clase abstracta llamada Base, que contiene los atributos comunes a todas las entidades: id, eliminado (para implementar baja lógica) y createdAt. A partir de esta clase, hicimos heredar a las cinco clases principales del sistema: Categoría, Producto, Usuario, Pedido y DetallePedido.

Cada una de estas clases incorpora sus propios atributos específicos. Por ejemplo, Categoría posee nombre y descripción; Producto incluye precio, stock y una referencia a su categoría; Usuario contiene los datos de contacto y el rol; y Pedido almacena la fecha, el estado, el total, la forma de pago, el usuario asociado y una lista de sus detalles.

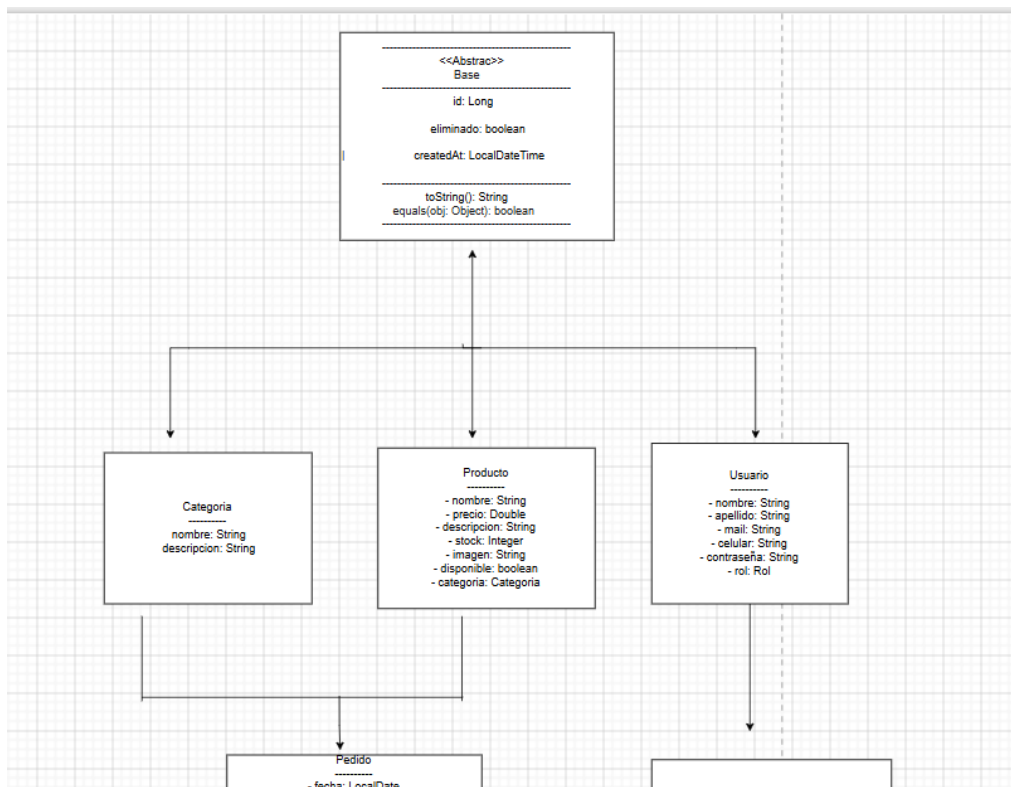
Para el cálculo del total del pedido, definimos la interfaz Calculable, que declara el método calcularTotal(). La clase Pedido implementa esta interfaz, y su método recorre todos los detalles del pedido para sumar sus subtotales, obteniendo así el monto final automáticamente.

En cuanto a las relaciones entre entidades, establecimos que:

- Una Categoría puede contener muchos Productos (relación 1 a N).
- Un Usuario puede realizar muchos Pedidos (relación 1 a N).
- Un Pedido se compone de varios DetallePedido (relación de composición 1 a N).
- Un Producto puede aparecer en muchos DetallePedido (relación 1 a N).

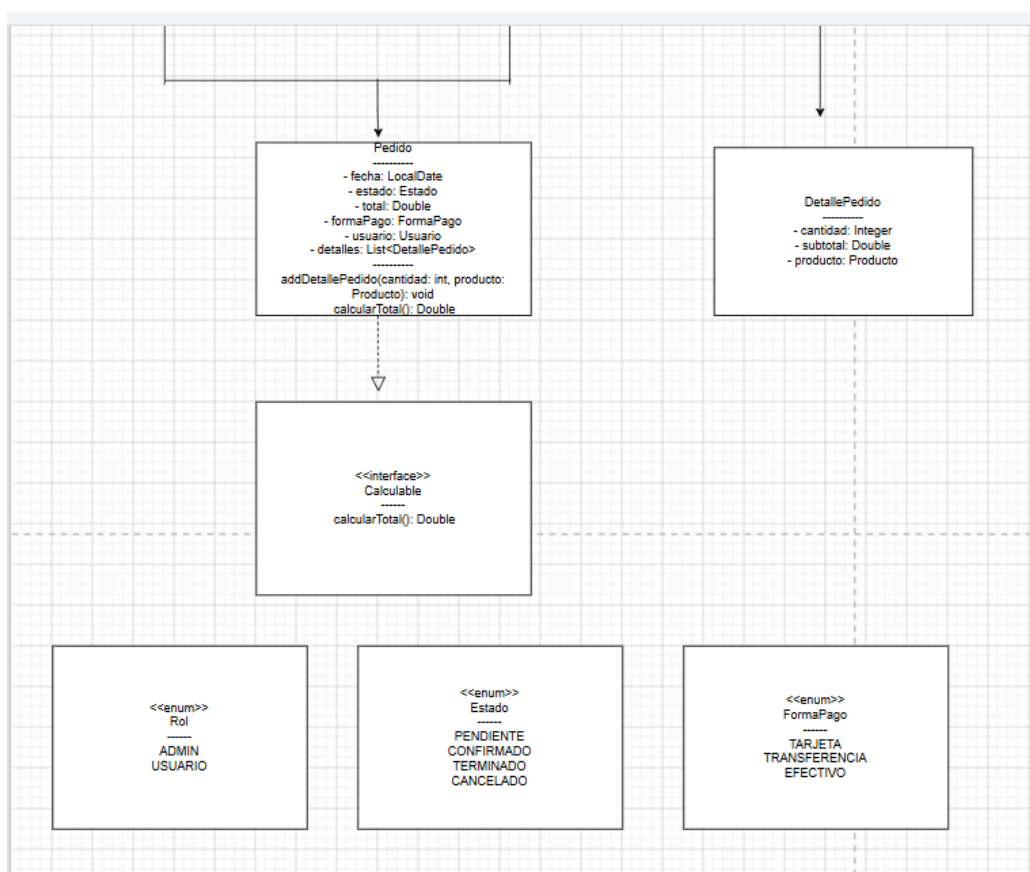
Finalmente, definimos tres enumerados para acotar los valores posibles de ciertos atributos: Rol (ADMIN y USUARIO), Estado (PENDIENTE, CONFIRMADO, TERMINADO y CANCELADO) y FormaPago (TARJETA, TRANSFERENCIA y EFECTIVO). Estos enums nos permiten mantener la consistencia de los datos y evitar errores al momento de cargar información en el sistema.

Este modelo conceptual fue la base sobre la cual implementamos todas las clases en Java, respetando las relaciones y asegurando que el código refleje fielmente el diseño planteado.



4.3. Decisiones de Diseño

- **Soft Delete (Baja Lógica):** Todas las entidades tienen el atributo eliminado (boolean). Al "eliminar", se marca como true en lugar de removerse de la colección. Esto permite mantener el historial y la integridad referencial.
- **Generación de IDs:** Se utiliza un contador estático en la clase Base para asignar IDs autoincrementales.
- **Validaciones:** Todas las validaciones ($\text{stock} \geq 0$, $\text{precio} \geq 0$, mail único, nombre no vacío) se realizan en los Services.
- **Uso de la interfaz Calculable:** La clase Pedido implementa Calculable y su método `calcularTotal()` recorre los detalles para sumar los subtotales.



-
- **Métodos específicos en Pedido:** Se implementaron `addDetallePedido()`, `findDetallePedidoByProducto()` y `deleteDetallePedidoByProducto()` según lo indicado en el UML.
 - **Menú en consola:** Cada opción del menú está validada con try-catch y maneja entradas incorrectas (números fuera de rango, textos donde se esperan números, etc.).

4.4. Capturas del Sistema Funcionando

Menú:

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===  
1. Gestión de Categorías  
2. Gestión de Productos  
3. Gestión de Usuarios  
4. Gestión de Pedidos  
0. Salir  
Seleccione una opción:
```

Menú – Categorías – Listar

```
0. Salir  
Seleccione una opción: 1  
  
--- GESTIÓN DE CATEGORÍAS ---  
1. Listar categorías  
2. Crear categoría  
3. Editar categoría  
4. Eliminar categoría  
0. Volver al menú principal  
Seleccione una opción: |
```

Creación de una Categoría - Solicita nombre y descripción, muestra el ID generado:

```
2. Crear categoría
3. Editar categoría
4. Eliminar categoría
0. Volver al menú principal
Seleccione una opción: 2
Nombre de la categoría: Tapeo
Descripción de la categoría: Pequeños platos a seleccionar con degustación

[ÉXITO] Categoría registrada. ID: 1
```

Menú Categoría – Editar: Solicita ID, muestra la categoría, permite modificar y confirmar

```
Seleccione una opción: 3

ID | NOMBRE | DESCRIPCIÓN
-----
1 | Tapeo | Pequeños platos a seleccionar con degustación

Ingrese el ID de la categoría a editar: 1
Nuevo nombre (Enter para omitir): Tapeos Varios
Nueva descripción (Enter para omitir): Platillos Variados para degustación

[ÉXITO] Categoría modificada correctamente.
```

Menú Categoría – Eliminar - Solicita ID y confirmación, marca como eliminado

```
--- GESTIÓN DE CATEGORÍAS ---
1. Listar categorías
2. Crear categoría
3. Editar categoría
4. Eliminar categoría
0. Volver al menú principal
Seleccione una opción: 4

ID | NOMBRE          | DESCRIPCIÓN
-----
1  | Tapeo           | Platillos Variados para degustación

Ingrese el ID de la categoría a eliminar: 1
¿Confirma la baja? (S/N): s

[ÉXITO] Categoría eliminada lógicamente.
```

Menú Productos – Listar - Muestra productos con ID, Nombre, Precio, Stock y Categoría:

```
--- GESTION DE PRODUCTOS ---
1. Listar productos
2. Crear producto
3. Editar producto
4. Eliminar producto
0. Volver al menú principal
Seleccione una opción: 1

ID | PRODUCTO          | PRECIO    | STOCK | CATEGORÍA
-----
1  | Pizza Muzzarella  | $16000,00 | 50    | Mundialísima
2  | Picada del Campeón | $30000,00 | 10    | Mundialísima
3  | Picada Mundialito  | $15000,00 | 15    | Mundialísima
4  | Segundo Francia   | $25000,00 | 6     | Mundialísima
```

Menú Productos – Crear - Solicita todos los datos y valida categoría existente:

```
Seleccione una opción: 2

-> AGREGAR NUEVO PRODUCTO AL CATÁLOGO
Nombre del producto: Pizza Muzzarella
Descripción: Pizza Mundialista
Precio unitario: 15.000
Stock inicial: 30
Nombre del archivo de imagen (ej: pizza.png): pizza.png

Categorías disponibles para asociar:
[2] - Mundialísima
Seleccione el ID de la categoría para este producto: 2

[ÉXITO] Producto registrado con el ID: 1
```

Menú – Productos – Editar - Permite actualizar campos y valida precio/stock:

```
4. Eliminar producto
0. Volver al menú principal
Seleccione una opción: 3

ID | PRODUCTO | PRECIO | STOCK | CATEGORÍA
-----
1 | Pizza Muzzarella | $15,00 | 30 | Mundialísima

Ingrese el ID del producto que desea modificar: 1
Nuevo nombre (Enter para omitir):
Nuevo precio (Enter para omitir): 16000
Nueva descripción (Enter para omitir):
Nuevo stock (Enter para omitir): 50
Nueva imagen (Enter para omitir):
¿Cambiar categoría? (S/N): S
Ingrese el ID de la nueva categoría: 2

[ÉXITO] Producto actualizado correctamente en el catálogo.
```

Menú Productos – Eliminar - Baja lógica con confirmación:

```
Seleccione una opción: 4

ID | PRODUCTO | PRECIO | STOCK | CATEGORÍA
-----
1 | Pizza Muzzarella | $16000,00 | 50 | Mundialísima
2 | Picada del Campeón | $30000,00 | 10 | Mundialísima
3 | Picada Mundialito | $15000,00 | 15 | Mundialísima
4 | Segundo Francia | $25000,00 | 6 | Mundialísima

Ingrese el ID del producto que desea dar de baja: 4
¿Seguro que desea ocultar este producto del catálogo? (S/N): S

[ÉXITO] Producto fue removido del catálogo.
```

Menú – Usuarios – Listar - Muestra ID, Nombre, Apellido, Mail, Rol:

```
Seleccione una opción: 1

ID | NOMBRE Y APELLIDO | E-MAIL | ROL
-----
1 | Nestor Gomez 0 | 0 | [ADMIN]
2 | Nestor Gomez | Nestorgomez@foodstruck.com.ar | [ADMIN]
3 | Victoria Lugo | victorialugo@foodstruck.com.ar | [USUARIO]
4 | Laura Dominguez | lauradom@foodstruck.com.ar | [USUARIO]
5 | Diego Maradona | diegoarmando@foodstruck.com.ar | [ADMIN]
6 | Kylian Mbappe | km@foodstruck.com.fr | [USUARIO]
```


Menú Usuarios – Crear - Solicita datos y valida mail único:

```
--- GESTION DE USUARIOS ---
1. Listar usuarios
2. Crear usuario
3. Editar usuario
4. Eliminar usuario
0. Volver al menú principal
Seleccione una opción: 2

-> REGISTRAR NUEVO USUARIO
Nombre: Segundo
Apellido: Francia
E-mail (Único): SF@foodstruck.com.ar
Celular: 222222222
Contraseña: 2
Seleccione el Rol:
  1. ADMIN
  2. USUARIO
Seleccione una opción: 2

[ÉXITO] Usuario registrado con el ID: 7
```

Menú Usuarios – Editar - Permite modificar y valida unicidad de mail:

```
Ingrese el ID del usuario que desea modificar: 1
Nuevo nombre (Enter para omitir): Nestor Armando
Nuevo apellido (Enter para omitir):
Nuevo e-mail (Enter para omitir):
Nuevo celular (Enter para omitir):
Nueva contraseña (Enter para omitir):
¿Cambiar el Rol? (S/N):
S
Seleccione nuevo Rol:
  1. ADMIN
  2. USUARIO
Seleccione una opción:
[ALERTA] Entrada inválida. Debe ingresar una opción numérica entera.
Seleccione una opción: 1

[ÉXITO] Datos de usuario actualizados. [cite: 351]
```

Menú Usuarios – Eliminar – Baja Lógica:

```
4. Eliminar usuario
0. Volver al menú principal
Seleccione una opción: 4

ID | NOMBRE Y APELLIDO | E-MAIL | ROL
-----
1 | Nestor Armando 0 | 0 | [ADMIN]
2 | Nestor Gomez | Nestorgomez@foodstruck.com.ar | [ADMIN]
3 | Victoria Lugo | victorialugo@foodstruck.com.ar | [USUARIO]
4 | Laura Dominguez | lauradom@foodstruck.com.ar | [USUARIO]
5 | Diego Maradona | diegoarmando@foodstruck.com.ar | [ADMIN]
6 | Kylian Mbappe | km@foodstruck.com.fr | [USUARIO]
7 | Segundo Francia | SF@foodstruck.com.ar | [USUARIO]

Ingrese el ID del usuario a eliminar: 1
¿Seguro que desea dar de baja a este usuario? (S/N): S

[ÉXITO] Usuario dado de baja lógicamente del sistema. [cite: 357]
```

Menú Pedidos – Listar - Muestra ID, Usuario, Estado, FormaPago, Total, Fecha:

```
Seleccione una opción: 1

=====
HISTORIAL GENERAL DE COMPRAS
=====
> PEDIDO #1 | Fecha: 2026-06-18 | Cliente: Nestor Gomez | Estado: PENDIENTE | Pago: EFECTIVO
-----
- Picada del Campeón x1 => Subtotal: $30000,00
>> TOTAL FINAL FACTURADO: $30000,00
=====

> PEDIDO #2 | Fecha: 2026-06-18 | Cliente: Laura Dominguez | Estado: PENDIENTE | Pago: EFECTIVO
-----
- Picada del Campeón x2 => Subtotal: $60000,00
- Picada Mundialito x1 => Subtotal: $15000,00
>> TOTAL FINAL FACTURADO: $75000,00
=====

> PEDIDO #3 | Fecha: 2026-06-18 | Cliente: Victoria Lugo | Estado: PENDIENTE | Pago: TRANSFERENCIA
-----
- Picada Mundialito x1 => Subtotal: $15000,00
```

Menú Pedidos – Crear - Selecciona usuario, agrega detalles, calcula total con Calculable:

```
[3] - Victoria Lugo
[4] - Laura Dominguez
[5] - Diego Maradona
[6] - Kylian Mbappe
[7] - Segundo Francia
ID del cliente: 2

Catálogo disponible:
[1] - Pizza Muzzarella | Precio: $16000,00 | Stock actual: 50
[2] - Picada del Campeón | Precio: $30000,00 | Stock actual: 10
[3] - Picada Mundialito | Precio: $15000,00 | Stock actual: 15
ID del producto a agregar: 2
Cantidad de unidades: 1
¿Desea añadir otro artículo al pedido? (S/N): n

Formas de pago aceptadas:
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
Seleccione una opción: 3

[ÉXITO] ;Pedido aprobado e ingresado a cocina!
```

Menú Pedidos – Actualizar - Cambia estado y forma de pago:

```
Seleccione una opción: 3

-> ACTUALIZAR INFORMACIÓN DE PEDIDO
Ingrese el ID del pedido a modificar: 3
¿Desea actualizar el Estado del envío? (S/N):
s
Estados válidos: 1. PENDIENTE | 2. CONFIRMADO | 3. TERMINADO | 4. CANCELADO [cite: 102]
Seleccione una opción: 4
¿Desea modificar la Forma de Pago? (S/N):

[ÉXITO] Los datos del pedido fueron actualizados. [cite: 389]
```

Menú Pedidos – Eliminar – Baja Lógica:

```
Seleccione una opción: 4

-> CANCELACIÓN DE PEDIDO (SOFT DELETE) [cite: 391]
Ingrese el ID del pedido que desea dar de baja: 2
¿Confirma la baja de la orden de compra? (S/N): [cite: 274]s

[ÉXITO] Orden removida de las pantallas de gestión activa. [cite: 395]
```

Validación de errores - Ejemplo: intentar crear producto con stock negativo, mostrar mensaje de error:

```
[7] - Segundo Francia
ID del cliente: 6

Catálogo disponible:
[1] - Pizza Muzzarella | Precio: $16000,00 | Stock actual: 50
[2] - Picada del Campeón | Precio: $30000,00 | Stock actual: 7
[3] - Picada Mundialito | Precio: $15000,00 | Stock actual: 13
ID del producto a agregar: 2
Cantidad de unidades: 10
¿Desea añadir otro artículo al pedido? (S/N): n

Formas de pago aceptadas:
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
Seleccione una opción: 1

[ERROR] TRANSACCIÓN ABORTADA: El pedido se canceló en memoria debido a: Falta de stock para 'Picada del Campeón'. Solicitado: 10 | Disponible:
```

Validación de errores - Ejemplo: intentar crear usuario con mail duplicado:

```
Seleccione una opción: 2

-> REGISTRAR NUEVO USUARIO
Nombre: falso
Apellido: Usuario
E-mail (Único): km@foodstruck.com.fr
Celular:
Contraseña:
Seleccione el Rol:
1. ADMIN
2. USUARIO
Seleccione una opción: 2

[ERROR] El e-mail 'km@foodstruck.com.fr' ya está registrado en el sistema.
```

Validación de errores - Ejemplo: intentar editar categoría inexistente:

```
Seleccione una opción: 3

ID | NOMBRE | DESCRIPCIÓN
-----
2 | Mundialísima | Menú Mundialista de los Campeones del Mundo

Ingrese el ID de la categoría a editar: 55
Nuevo nombre (Enter para omitir):
Nueva descripción (Enter para omitir):

[ERROR] No se encontró ninguna categoría con el ID: 55
```

5 – Dificultades y Soluciones

Dificultad: Inicialización de Servicios con Dependencias

Problema:

En la clase Main, al declarar los servicios como variables estáticas en el orden mostrado, surgió un error porque PedidoService necesitaba usuarioService y productoService para funcionar, pero estos aún no estaban inicializados cuando se creaba PedidoService.

Esto provocaba que al ejecutar el programa, PedidoService recibiera null en sus dependencias y lanzara un NullPointerException.

Solución:

Se movió la inicialización de todos los servicios dentro del método main(), garantizando que usuarioService y productoService se creen antes que pedidoService. De esta manera, las dependencias llegan correctamente al constructor.

6. BIBLIOGRAFÍA / WEBGRAFÍA

- <https://www.youtube.com/watch?v=voMOPgtnJto&list=PLOw7b-NX043aSC7ZNtEuVfY8xZoNzVqdJ>
-

7. ANEXOS

7.1. Link al Repositorio GitHub

7.2. Link al Video Demostrativo